U.PORTO

FEUP FACULDADE DE ENGENHARIA
UNIVERSIDADE DO PORTO

Second Project

FTP Application & Network Study and Setup

Authors:

Manoela Blanke Américo | up202201391

Marcel Segundo Medeiros | up202200041

Course Unit: Computer Networks

Class and Group: 3LEIC10 - Group 3

Academic Year: 2024/2025

Index

Summary **2**
Introduction **2**
Architecture **3**
Code Structure **3**
Main Use Cases **5**
Logical Link Protocol **5**
Application Protocol **6**
1. Initialization with applicationLayer() 6
2. File Transmission: SendFile() 7
3. File Reception: ReceiveFile() 7
Validation **7**
Data Link Protocol Efficiency **7**
1. Other important statistics (showed at the end of each execution) 8
Conclusion **8**

Summary

The lab involves developing a basic FTP client in C and configuring networks. In Part 1, the goal is to create an FTP client following RFCs 959 and 1738, capable of connecting to a server, authenticating, using passive mode, downloading a file, and saving it locally or indicating failure, addressing challenges like using sockets, DNS, and implementing control and data connections. The objective is to understand the FTP protocol, client-server concepts, and TCP/IP programming in C. In Part 2, the focus is on configuring networks with subnets (e.g., 172.16.30.0/24, 172.16.31.0/24), NAT, routers, and hosts, and show that our FTP download application works for the specific network setup that has during the classes.

Introduction

This project focuses on two main parts: the development of a download application and the configuration of a network environment. The first part aims to implement an FTP client application capable of parsing a URL, creating a socket, logging into the server, navigating directories, establishing a passive connection, and downloading a specified file. The steps include parsing the URL, creating a socket for server communication, sending commands for login and directory navigation, handling the passive mode response to extract a new IP and port, creating a data socket, and retrieving the desired file.

The second part centers on the configuration and study of a network environment through a series of experiments. These include configuring IP addresses and testing connectivity within a subnet, implementing bridges in a switch, configuring a Linux-based router, setting up NAT on a commercial router, exploring DNS functionality, and establishing TCP connections. Each experiment aims to deepen understanding of key networking concepts and protocols.

The report details the implementation process for each part, followed by validation through testing to ensure the proper functionality of the FTP client and the configured network. The conclusions provide insights into the outcomes, challenges, and lessons learned during the project.

Part 1 - Download application

1. Architecture

The code implements an FTP client application designed to handle file downloads from a server. It employs a modular structure, dividing responsibilities across different functional components to ensure clarity and maintainability. The application workflow is driven by sequential operations that involve parsing the URL, managing sockets for control and data connections, and executing FTP commands.

```
gcc -W ftp.c aux.c -o ftp        #compile

./ftp ftp://[user:password@]<host>/<url-path>        #run
```

2. Code Structure

The code is divided into two main files (that use some code from the provided files getip.c and clientTCP.c) - aux.c and ftp.c

aux.c

- **`get_ip(char* hostname)`** - Resolves the hostname to an IP address using DNS and returns the IP in string format.
- **`create_socket(const char* ip, int port)`** - Creates a socket and establishes a TCP connection to the server at the specified IP and port.
- **`send_socket(int sockfd, char* message, char* header)`** - Sends a message in FTP format (command and argument) through the provided socket.
- **`parse_url(struct components* c, char* url)`** - Parses an FTP URL and extracts its components: username, password, hostname, path, and filename.
- **`read_socket(int sockfd, char* response, size_t response_size)`** - Reads the server's response through the socket and displays FTP protocol messages.
- **`parse_pasv_response(const char* response, char* ip, int* port)`** - Analyzes the PASV command response to extract the server's IP and port for data transfer.
- **`login(int sockfd, struct components c, char* response)`** - Sends the login commands (USER and PASS) and validates the server's responses.
- **`send_retr_command(int sockfd, const char* filename, char* response, size_t response_size)`** - Sends the RETR command to download a file from the server.
- **`download_file(int sockfd, const char* local_filename)`** - Receives the file data through the data socket and saves it to the specified local file.
- **`close_socket(int sockfd)`** - Closes an active socket safely.

ftp.c

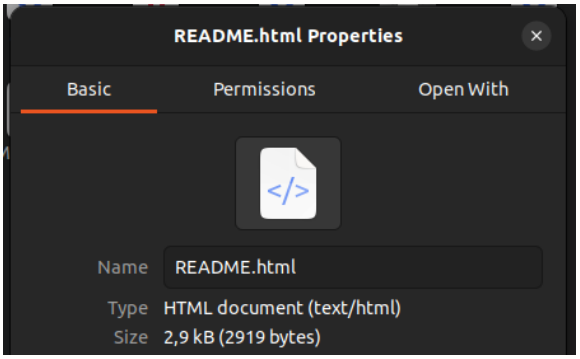
- **`main()`** - Executes the application and calls the functions above.

3. Results

3.1 Download of README.html in mirrors.up.pt/debian (Experiment of the first class)

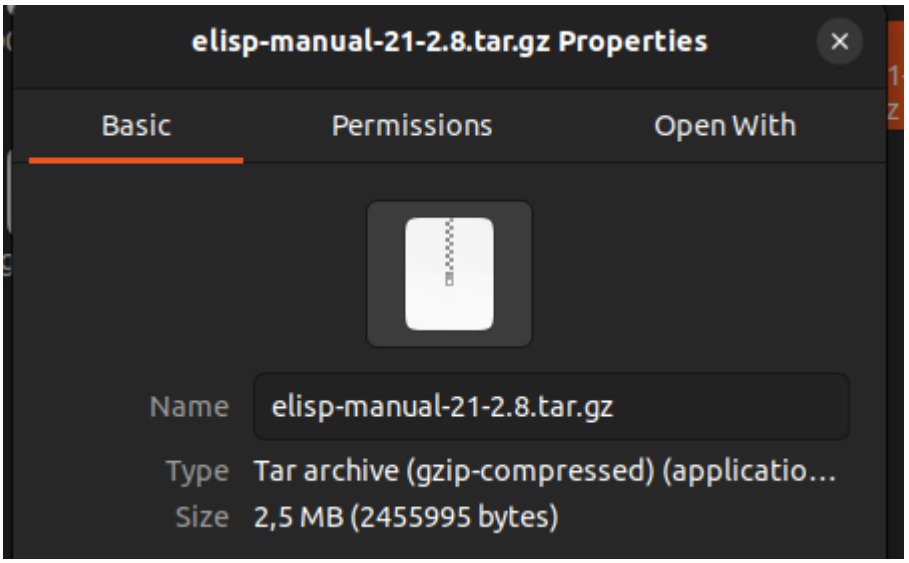
```
→  RCOM-L2 git:(main) ✗ ./ftp ftp://mirrors.up.pt/debian/README.html
Connection estabilished
220-Welcome to the University of Porto's mirror archive (mirrors.up.pt)
220----------------------------------------------------------------
220-
220-All connections and transfers are logged. The max number of connections i
220-
220-For more information please visit our website: http://mirrors.up.pt/
220-Questions and comments can be sent to mirrors@uporto.pt
220-
220-
220
USER anonymous

331 Please specify the password.
PASS anonymous

230 Login successful.
CWD debian

250 Directory successfully changed.
pasv

227 Entering Passive Mode (193,137,29,15,200,19).
Connection estabilished
Sending RETR command
RETR README.html

Downloading file README.html
Downloading file README.html
Downloading file README.html
Downloading file README.html
File downloaded successfully as README.html
```

Output

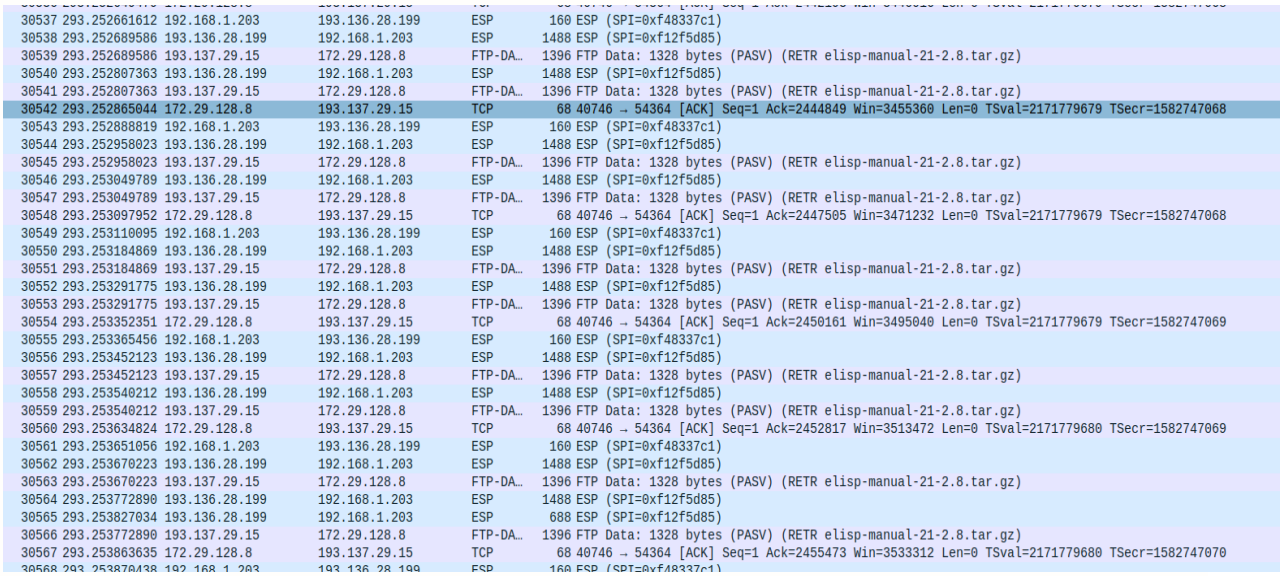


File Downloaded

3.2 Download of ftp.up.pt/pub/gnu/emacs/elisp-manual-21-2.8.tar.gz



File Downloaded



30537	293.252661612	192.168.1.203	193.136.28.199	ESP	160	ESP (SPI=0xf48337c1)
30538	293.252689586	193.136.28.199	192.168.1.203	ESP	1488	ESP (SPI=0xf12f5d85)
30539	293.252689586	193.137.29.15	172.29.128.8	FTP-DA…	1396	FTP Data: 1328 bytes (PASV) (RETR elisp-manual-21-2.8.tar.gz)
30540	293.252807363	193.136.28.199	192.168.1.203	ESP	1488	ESP (SPI=0xf12f5d85)
30541	293.252807363	193.137.29.15	172.29.128.8	FTP-DA…	1396	FTP Data: 1328 bytes (PASV) (RETR elisp-manual-21-2.8.tar.gz)
30542	293.252865044	172.29.128.8	193.137.29.15	TCP	68	40746 → 54364 [ACK] Seq=1 Ack=2444849 Win=3455360 Len=0 TSval=2171779679 TSecr=1582747068
30543	293.252888819	192.168.1.203	193.136.28.199	ESP	160	ESP (SPI=0xf48337c1)
30544	293.252958023	193.136.28.199	192.168.1.203	ESP	1488	ESP (SPI=0xf12f5d85)
30545	293.252958023	193.137.29.15	172.29.128.8	FTP-DA…	1396	FTP Data: 1328 bytes (PASV) (RETR elisp-manual-21-2.8.tar.gz)
30546	293.253049789	193.136.28.199	192.168.1.203	ESP	1488	ESP (SPI=0xf12f5d85)
30547	293.253049789	193.137.29.15	172.29.128.8	FTP-DA…	1396	FTP Data: 1328 bytes (PASV) (RETR elisp-manual-21-2.8.tar.gz)
30548	293.253097952	172.29.128.8	193.137.29.15	TCP	68	40746 → 54364 [ACK] Seq=1 Ack=2447505 Win=3471232 Len=0 TSval=2171779679 TSecr=1582747068
30549	293.253110095	192.168.1.203	193.136.28.199	ESP	160	ESP (SPI=0xf48337c1)
30550	293.253184869	193.136.28.199	192.168.1.203	ESP	1488	ESP (SPI=0xf12f5d85)
30551	293.253184869	193.137.29.15	172.29.128.8	FTP-DA…	1396	FTP Data: 1328 bytes (PASV) (RETR elisp-manual-21-2.8.tar.gz)
30552	293.253291775	193.136.28.199	192.168.1.203	ESP	1488	ESP (SPI=0xf12f5d85)
30553	293.253291775	193.137.29.15	172.29.128.8	FTP-DA…	1396	FTP Data: 1328 bytes (PASV) (RETR elisp-manual-21-2.8.tar.gz)
30554	293.253352351	172.29.128.8	193.137.29.15	TCP	68	40746 → 54364 [ACK] Seq=1 Ack=2450161 Win=3495040 Len=0 TSval=2171779679 TSecr=1582747069
30555	293.253365456	192.168.1.203	193.136.28.199	ESP	160	ESP (SPI=0xf48337c1)
30556	293.253452123	193.136.28.199	192.168.1.203	ESP	1488	ESP (SPI=0xf12f5d85)
30557	293.253452123	193.137.29.15	172.29.128.8	FTP-DA…	1396	FTP Data: 1328 bytes (PASV) (RETR elisp-manual-21-2.8.tar.gz)
30558	293.253540212	193.136.28.199	192.168.1.203	ESP	1488	ESP (SPI=0xf12f5d85)
30559	293.253540212	193.137.29.15	172.29.128.8	FTP-DA…	1396	FTP Data: 1328 bytes (PASV) (RETR elisp-manual-21-2.8.tar.gz)
30560	293.253634824	172.29.128.8	193.137.29.15	TCP	68	40746 → 54364 [ACK] Seq=1 Ack=2452817 Win=3513472 Len=0 TSval=2171779680 TSecr=1582747069
30561	293.253651056	192.168.1.203	193.136.28.199	ESP	160	ESP (SPI=0xf48337c1)
30562	293.253670223	193.136.28.199	192.168.1.203	ESP	1488	ESP (SPI=0xf12f5d85)
30563	293.253670223	193.137.29.15	172.29.128.8	FTP-DA…	1396	FTP Data: 1328 bytes (PASV) (RETR elisp-manual-21-2.8.tar.gz)
30564	293.253772890	193.136.28.199	192.168.1.203	ESP	1488	ESP (SPI=0xf12f5d85)
30565	293.253827034	193.136.28.199	192.168.1.203	ESP	688	ESP (SPI=0xf12f5d85)
30566	293.253772890	193.137.29.15	172.29.128.8	FTP-DA…	1396	FTP Data: 1328 bytes (PASV) (RETR elisp-manual-21-2.8.tar.gz)
30567	293.253863635	172.29.128.8	193.137.29.15	TCP	68	40746 → 54364 [ACK] Seq=1 Ack=2455473 Win=3533312 Len=0 TSval=2171779680 TSecr=1582747070
30568	293.253870438	192.168.1.203	193.136.28.199	ESP	160	ESP (SPI=0xf48337c1)

Wireshark Capture of FTP packets

Part 2 - Network configuration and analysis

1. Configure an IP Network

We connected the network interfaces (E1) of two machines, **tuxY3** and **tuxY4**, to a switch. This allows them to communicate with each other over the network. The configuration of the network interfaces was done with the command ifconfig.

ifconfig eth1 172.16.30.1/24 (TUX3)

ifconfig eth1 172.16.30.254/24 (TUX4)

When this configuration is done, TUX3 and TUX4 are in the same network (172.16.30.0/24) and they can ping each other:

17	15.200002705	172.16.30.1	172.16.30.254	ICMP	98	Echo (ping) request id=0x0bab, seq=2/512, ttl=64 (reply in 18)
18	15.200157199	172.16.30.254	172.16.30.1	ICMP	98	Echo (ping) reply id=0x0bab, seq=2/512, ttl=64 (request in 17)
19	16.224000990	172.16.30.1	172.16.30.254	ICMP	98	Echo (ping) request id=0x0bab, seq=3/768, ttl=64 (reply in 20)
20	16.224175179	172.16.30.254	172.16.30.1	ICMP	98	Echo (ping) reply id=0x0bab, seq=3/768, ttl=64 (request in 19)
21	16.543843912	Routerbo_1c:9f:5b	Spanning-tree-(for-…	STP	60	RST. Root = 32768/0/c4:ad:34:1c:9f:5b Cost = 0 Port = 0x0001
22	17.248003487	172.16.30.1	172.16.30.254	ICMP	98	Echo (ping) request id=0x0bab, seq=4/1024, ttl=64 (reply in 23)
23	17.248155326	172.16.30.254	172.16.30.1	ICMP	98	Echo (ping) reply id=0x0bab, seq=4/1024, ttl=64 (request in 22)
24	18.272018577	172.16.30.1	172.16.30.254	ICMP	98	Echo (ping) request id=0x0bab, seq=5/1280, ttl=64 (reply in 25)
25	18.272170137	172.16.30.254	172.16.30.1	ICMP	98	Echo (ping) reply id=0x0bab, seq=5/1280, ttl=64 (request in 24)
26	18.545972547	Routerbo_1c:9f:5b	Spanning-tree-(for-…	STP	60	RST. Root = 32768/0/c4:ad:34:1c:9f:5b Cost = 0 Port = 0x0001
27	19.296025727	172.16.30.1	172.16.30.254	ICMP	98	Echo (ping) request id=0x0bab, seq=6/1536, ttl=64 (reply in 28)
28	19.296186996	172.16.30.254	172.16.30.1	ICMP	98	Echo (ping) reply id=0x0bab, seq=6/1536, ttl=64 (request in 27)
29	20.320000422	172.16.30.1	172.16.30.254	ICMP	98	Echo (ping) request id=0x0bab, seq=7/1792, ttl=64 (reply in 30)
30	20.320156591	172.16.30.254	172.16.30.1	ICMP	98	Echo (ping) reply id=0x0bab, seq=7/1792, ttl=64 (request in 29)
31	20.548115024	Routerbo_1c:9f:5b	Spanning-tree-(for-…	STP	60	RST. Root = 32768/0/c4:ad:34:1c:9f:5b Cost = 0 Port = 0x0001
32	21.344004052	172.16.30.1	172.16.30.254	ICMP	98	Echo (ping) request id=0x0bab, seq=8/2048, ttl=64 (reply in 33)
33	21.344180407	172.16.30.254	172.16.30.1	ICMP	98	Echo (ping) reply id=0x0bab, seq=8/2048, ttl=64 (request in 32)
34	22.368021813	172.16.30.1	172.16.30.254	ICMP	98	Echo (ping) request id=0x0bab, seq=9/2304, ttl=64 (reply in 35)
35	22.368175259	172.16.30.254	172.16.30.1	ICMP	98	Echo (ping) reply id=0x0bab, seq=9/2304, ttl=64 (request in 34)
36	22.550318905	Routerbo_1c:9f:5b	Spanning-tree-(for-…	STP	60	RST. Root = 32768/0/c4:ad:34:1c:9f:5b Cost = 0 Port = 0x0001
37	23.392024509	172.16.30.1	172.16.30.254	ICMP	98	Echo (ping) request id=0x0bab, seq=10/2560, ttl=64 (reply in 38)
38	23.392178793	172.16.30.254	172.16.30.1	ICMP	98	Echo (ping) reply id=0x0bab, seq=10/2560, ttl=64 (request in 37)
39	24.416024642	172.16.30.1	172.16.30.254	ICMP	98	Echo (ping) request id=0x0bab, seq=11/2816, ttl=64 (reply in 40)
40	24.416180952	172.16.30.254	172.16.30.1	ICMP	98	Echo (ping) reply id=0x0bab, seq=11/2816, ttl=64 (request in 39)
41	24.552468530	Routerbo_1c:9f:5b	Spanning-tree-(for-…	STP	60	RST. Root = 32768/0/c4:ad:34:1c:9f:5b Cost = 0 Port = 0x0001
42	25.440021095	172.16.30.1	172.16.30.254	ICMP	98	Echo (ping) request id=0x0bab, seq=12/3072, ttl=64 (reply in 43)
43	25.440176706	172.16.30.254	172.16.30.1	ICMP	98	Echo (ping) reply id=0x0bab, seq=12/3072, ttl=64 (request in 42)

In TUX3, ping TUX4:

Src IP	Src MAC	Dst IP	Dst MAC
TUX3 IP	TUX3 MAC	TUX4 IP	TUX4 MAC

ARP (Address Resolution Protocol) packets map IP addresses to MAC addresses on a local network. When tuxY3 pings tuxY4, it sends an ARP request to learn the MAC address of tuxY4. tuxY4 responds with its MAC address, allowing communication. The **`ping`** command generates ICMP Echo Request and ICMP Echo Reply packets.

To identify if an Ethernet frame is ARP, IP, or ICMP, check the **EtherType field** in the Ethernet header: `0x0806` indicates ARP, while `0x0800` indicates IP. For IP frames, examine the **Protocol field** in the IP header: 1 represents ICMP. Tools like Wireshark simplify this process by decoding these fields automatically.

2. Implement Two Bridges in a Switch

After the IP configuration of TUX3 and TUX4, we configured TUX2 in another network (172.16.31.0/24) with IP 172.16.31.1. With this set, two bridges (bridge30 and bridge31) have been created. Bridges connect and filter traffic between different network segments at the data link layer, improving network efficiency and allowing devices in separate segments to communicate as if on the same network. After that, TUX3 and TUX4 still have connection and TUX2 is unreachable.

We've opened the switch console in GTK Term and applied the following commands to configure bridges:

/interface bridge add name=bridge30
/interface bridge add name=bridge31

To remove the TUX's ethernets from the default bridge and add them to our custom bridges, the following commands were used:

/interface bridge port remove [find interface =etherX]
/interface bridge port add bridge=bridgeXX interface=etherX

(In etherX, assume that X is the corresponding ethernet of each TUX in the switch. For bridgeXX, assume that XX is 30 or 31).

3. Configure a Router in Linux

In the third experience, we had the opportunity to make TUX4 become a router. Our main goal in this step was to find a way to ping Tux2 and Tux3, even in different networks. This was done by adding the eth2 of TUX4 to the network 172.16.31.0/24, and then adding it to bridge31. Then, we've added the route to the neighbour network with the correct gateways (recheable IP of TUX4) in TUX3 and TUX2.

After that, the first step is enabling IP forwarding, a critical feature that allows the system to function as a router. To further enhance network functionality, it is necessary to disable the default setting that ignores ICMP echo broadcast requests. In this experience, we've used the commands below:

ifconfig eth2 172.16.31.253/24 (eth2 of TUX4)
/interface bridge port remove [find interface =etherX]
/interface bridge port add bridge=bridge31 interface=etherX (add eth2 of TUX4 to the bridge)
route add -net 172.16.31.0/24 gw 172.16.30.254 (in TUX3)
route add -net 172.16.30.0/24 gw 172.16.31.253 (in TUX2)
sysctl net.ipv4.ip_forward=1 (in TUX4)
sysctl net.ipv4.icmp_echo_ignore_broadcasts=0 (in TUX4)

107	162.949606148	172.16.30.1	172.16.31.1	ICMP	98	Echo (ping) request id=0x1962, seq=8/2048, ttl=63 (reply in 108)
108	162.949753375	172.16.31.1	172.16.30.1	ICMP	98	Echo (ping) reply id=0x1962, seq=8/2048, ttl=64 (request in 107)
109	163.973614473	172.16.30.1	172.16.31.1	ICMP	98	Echo (ping) request id=0x1962, seq=9/2304, ttl=63 (reply in 110)
110	163.973763795	172.16.31.1	172.16.30.1	ICMP	98	Echo (ping) reply id=0x1962, seq=9/2304, ttl=64 (request in 109)
111	164.169103486	Routerbo_1c:9f:65	Spanning-tree-(for-…	STP	60	RST. Root = 32768/0/c4:ad:34:1c:9f:59 Cost = 0 Port = 0x0002
112	164.997597030	172.16.30.1	172.16.31.1	ICMP	98	Echo (ping) request id=0x1962, seq=10/2560, ttl=63 (reply in 113)
113	164.997768841	172.16.31.1	172.16.30.1	ICMP	98	Echo (ping) reply id=0x1962, seq=10/2560, ttl=64 (request in 112
114	166.021583292	172.16.30.1	172.16.31.1	ICMP	98	Echo (ping) request id=0x1962, seq=11/2816, ttl=63 (reply in 115)
115	166.021736874	172.16.31.1	172.16.30.1	ICMP	98	Echo (ping) reply id=0x1962, seq=11/2816, ttl=64 (request in 114
116	166.171187139	Routerbo_1c:9f:65	Spanning-tree-(for-…	STP	60	RST. Root = 32768/0/c4:ad:34:1c:9f:59 Cost = 0 Port = 0x0002
117	167.045576961	172.16.30.1	172.16.31.1	ICMP	98	Echo (ping) request id=0x1962, seq=12/3072, ttl=63 (reply in 118)
118	167.045721534	172.16.31.1	172.16.30.1	ICMP	98	Echo (ping) reply id=0x1962, seq=12/3072, ttl=64 (request in 117
119	168.069581181	172.16.30.1	172.16.31.1	ICMP	98	Echo (ping) request id=0x1962, seq=13/3328, ttl=63 (reply in 120)
120	168.069726731	172.16.31.1	172.16.30.1	ICMP	98	Echo (ping) reply id=0x1962, seq=13/3328, ttl=64 (request in 119
121	168.173277583	Routerbo_1c:9f:65	Spanning-tree-(for-…	STP	60	RST. Root = 32768/0/c4:ad:34:1c:9f:59 Cost = 0 Port = 0x0002
122	169.093584845	172.16.30.1	172.16.31.1	ICMP	98	Echo (ping) request id=0x1962, seq=14/3584, ttl=63 (reply in 123)
123	169.093729278	172.16.31.1	172.16.30.1	ICMP	98	Echo (ping) reply id=0x1962, seq=14/3584, ttl=64 (request in 122
124	170.121575091	172.16.30.1	172.16.31.1	ICMP	98	Echo (ping) request id=0x1962, seq=15/3840, ttl=63 (reply in 125)
125	170.121749137	172.16.31.1	172.16.30.1	ICMP	98	Echo (ping) reply id=0x1962, seq=15/3840, ttl=64 (request in 124
126	170.175345412	Routerbo_1c:9f:65	Spanning-tree-(for-…	STP	60	RST. Root = 32768/0/c4:ad:34:1c:9f:59 Cost = 0 Port = 0x0002
127	171.042584272	0.0.0.0	255.255.255.255	MNDP	183	5678 → 5678 Len=141
128	171.042609275	Routerbo_1c:9f:65	CDP/VTP/DTP/PAgP/UD…	CDP	117	Device ID: MikroTik Port ID: bridge31/ether24
129	171.042690222	Routerbo_1c:9f:65	LLDP_Multicast	LLDP	134	MA/c4:ad:34:1c:9f:4e IN/bridge31/ether24 120 SysN=MikroTik SysD=Mik
130	171.141561595	172.16.30.1	172.16.31.1	ICMP	98	Echo (ping) request id=0x1962, seq=16/4096, ttl=63 (reply in 131)
131	171.141705819	172.16.31.1	172.16.30.1	ICMP	98	Echo (ping) reply id=0x1962, seq=16/4096, ttl=64 (request in 130
132	172.165666472	172.16.30.1	172.16.31.1	ICMP	98	Echo (ping) request id=0x1962, seq=17/4352, ttl=63 (reply in 133)
133	172.165816423	172.16.31.1	172.16.30.1	ICMP	98	Echo (ping) reply id=0x1962, seq=17/4352, ttl=64 (request in 132
134	172.176783351	Routerbo_1c:9f:65	Spanning-tree-(for-…	STP	60	RST. Root = 32768/0/c4:ad:34:1c:9f:59 Cost = 0 Port = 0x0002

Tux2 ping Tux3

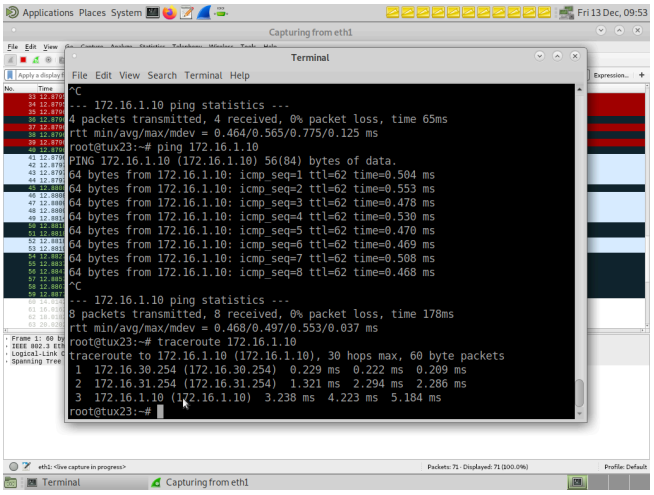
4. Configure a Commercial Router and Implement NAT

In the fourth experience, we configured a commercial router and implemented NAT to enable communication between devices across different subnets. The RC router's ether1 interface was connected to the lab network on PY.12 (with NAT enabled by default), and its ether2 interface was linked to a port on bridge31. IP addresses were configured via the router's serial console, and routes were added in tuxY3, tuxY4, tuxY2, and RC to ensure connectivity across subnets 172.16.30.0/24, 172.16.31.0/24, and 172.16.1.0/24.

Further, in tuxY2, we adjusted settings to disable the acceptance of ICMP redirects and modified the routes to use RC as the gateway to the 172.16.30.0/24 subnet instead of tuxY4. This allowed us to observe the path of ICMP ECHO and ECHO-REPLY packets through MAC address analysis.

Lasty, the connectivity was verified by pinging devices and analyzing traffic using Wireshark. Additionally, the NAT functionality was tested by pinging the FTP server (172.16.1.10) from tuxY3, with and without NAT enabled, to observe its impact on connectivity. The commands used in this experience were:

route add -net 172.16.1.0/24 gw 172.16.30.254(in tuxY3)
route add -net 172.16.1.0/24 gw 172.16.31.254(in tuxY4)
route add -net 172.16.1.0/24 gw 172.16.31.254(in tuxY2)
route add -net 172.16.30.0/24 gw 172.16.31.253(in RC)
sysctl net.ipv4.conf.eth1.accept_redirects=0
sysctl net.ipv4.conf.all.accept_redirects=0
ping 172.16.1.10 (from tuxY3)



TraceRoute Netlab

	Time	Source	Destination	Protocol	Length	Info
→	1 0.000000000	172.16.31.1	172.16.1.10	ICMP	98	Echo (ping) request id=0x0dc0, seq=6/1536, ttl=64 (reply in 2)
←	2 0.000350882	172.16.1.10	172.16.31.1	ICMP	98	Echo (ping) reply id=0x0dc0, seq=6/1536, ttl=63 (request in 1)
	3 0.095882496	3Com_a0:f9:60	Routerboardc_ea:74:…	ARP	42	Who has 172.16.31.254? Tell 172.16.31.1
	4 0.096014846	Routerboardc_ea:74:…	3Com_a0:f9:60	ARP	60	172.16.31.254 is at 74:4d:28:ea:74:a5
	5 1.137558000	Routerboardc_1c:9f:…	Spanning-tree-(for-…	STP	60	RST. Root = 32768/0/74:4d:28:ea:74:a5 Cost = 10 Port = 0x0001
	6 1.024051612	172.16.31.1	172.16.1.10	ICMP	98	Echo (ping) request id=0x0dc0, seq=7/1792, ttl=64 (reply in 7)
	7 1.024353676	172.16.1.10	172.16.31.1	ICMP	98	Echo (ping) reply id=0x0dc0, seq=7/1792, ttl=63 (request in 6)
	8 2.047918631	172.16.31.1	172.16.1.10	ICMP	98	Echo (ping) request id=0x0dc0, seq=8/2048, ttl=64 (reply in 9)
	9 2.048264624	172.16.1.10	172.16.31.1	ICMP	98	Echo (ping) reply id=0x0dc0, seq=8/2048, ttl=63 (request in 8)
	10 2.139681400	Routerboardc_1c:9f:…	Spanning-tree-(for-…	STP	60	RST. Root = 32768/0/74:4d:28:ea:74:a5 Cost = 10 Port = 0x0001
	11 2.618351676	fe80::201:2ff:fea0:…	ff02::fb	MDNS	180	Standard query 0x0000 PTR _ftp._tcp.local, "QM" question PTR _nfs._tcp.local, "QM" question PTR _afpovertcp._tcp.local, "QM"
	12 2.618420679	172.16.31.1	224.0.0.251	MDNS	160	Standard query 0x0000 PTR _ftp._tcp.local, "QM" question PTR _nfs._tcp.local, "QM" question PTR _afpovertcp._tcp.local, "QM"
	13 3.071919183	172.16.31.1	172.16.1.10	ICMP	98	Echo (ping) request id=0x0dc0, seq=9/2304, ttl=64 (reply in 14)
	14 3.072234935	172.16.1.10	172.16.31.1	ICMP	98	Echo (ping) reply id=0x0dc0, seq=9/2304, ttl=63 (request in 13)
	15 4.095905414	172.16.31.1	172.16.1.10	ICMP	98	Echo (ping) request id=0x0dc0, seq=10/2560, ttl=64 (reply in 16)
	16 4.096253223	172.16.1.10	172.16.31.1	ICMP	98	Echo (ping) reply id=0x0dc0, seq=10/2560, ttl=63 (request in 15)
	17 4.141875008	Routerboardc_1c:9f:…	Spanning-tree-(for-…	STP	60	RST. Root = 32768/0/74:4d:28:ea:74:a5 Cost = 10 Port = 0x0001
	18 5.119906169	172.16.31.1	172.16.1.10	ICMP	98	Echo (ping) request id=0x0dc0, seq=11/2816, ttl=64 (reply in 19)
	19 5.120195661	172.16.1.10	172.16.31.1	ICMP	98	Echo (ping) reply id=0x0dc0, seq=11/2816, ttl=63 (request in 18)
	20 6.143903219	172.16.31.1	172.16.1.10	ICMP	98	Echo (ping) request id=0x0dc0, seq=12/3072, ttl=64 (reply in 22)
	21 6.143960978	Routerboardc_1c:9f:…	Spanning-tree-(for-…	STP	60	RST. Root = 32768/0/74:4d:28:ea:74:a5 Cost = 10 Port = 0x0001
	22 6.144247956	172.16.1.10	172.16.31.1	ICMP	98	Echo (ping) reply id=0x0dc0, seq=12/3072, ttl=63 (request in 20)
	23 7.167896565	172.16.31.1	172.16.1.10	ICMP	98	Echo (ping) request id=0x0dc0, seq=13/3328, ttl=64 (reply in 24)
	24 7.168229009	172.16.1.10	172.16.31.1	ICMP	98	Echo (ping) reply id=0x0dc0, seq=13/3328, ttl=63 (request in 23)
	25 8.140102748	Routerboardc_1c:9f:…	Spanning-tree-(for-…	STP	60	RST. Root = 32768/0/74:4d:28:ea:74:a5 Cost = 10 Port = 0x0001
	26 8.191913095	172.16.31.1	172.16.1.10	ICMP	98	Echo (ping) request id=0x0dc0, seq=14/3584, ttl=64 (reply in 27)
	27 8.192255597	172.16.1.10	172.16.31.1	ICMP	98	Echo (ping) reply id=0x0dc0, seq=14/3584, ttl=63 (request in 26)

Tux2 ping NetLab

	Time	Source	Destination	Protocol	Length	Info
	1 0.000000000	Routerboardc_1c:9f:…	Spanning-tree-(for-…	STP	60	RST. Root = 32768/0/c4:ad:34:1c:9f:5b Cost = 0 Port = 0x0001
	2 0.737998010	172.16.30.1	172.16.1.10	ICMP	98	Echo (ping) request id=0x0d9c, seq=4/1024, ttl=64 (reply in 3)
	3 0.738517577	172.16.1.10	172.16.30.1	ICMP	98	Echo (ping) reply id=0x0d9c, seq=4/1024, ttl=62 (request in 2)
	4 1.752030654	172.16.30.1	172.16.1.10	ICMP	98	Echo (ping) request id=0x0d9c, seq=5/1280, ttl=64 (reply in 5)
	5 1.752458236	172.16.1.10	172.16.30.1	ICMP	98	Echo (ping) reply id=0x0d9c, seq=5/1280, ttl=62 (request in 4)
	6 2.002117195	Routerboardc_1c:9f:…	Spanning-tree-(for-…	STP	60	RST. Root = 32768/0/c4:ad:34:1c:9f:5b Cost = 0 Port = 0x0001
	7 2.776036210	172.16.30.1	172.16.1.10	ICMP	98	Echo (ping) request id=0x0d9c, seq=6/1536, ttl=64 (reply in 8)
	8 2.776488866	172.16.1.10	172.16.30.1	ICMP	98	Echo (ping) reply id=0x0d9c, seq=6/1536, ttl=62 (request in 7)
	9 3.800031443	172.16.30.1	172.16.1.10	ICMP	98	Echo (ping) request id=0x0d9c, seq=7/1792, ttl=64 (reply in 10)
	10 3.800511198	172.16.1.10	172.16.30.1	ICMP	98	Echo (ping) reply id=0x0d9c, seq=7/1792, ttl=62 (request in 9)
	11 4.004237657	Routerboardc_1c:9f:…	Spanning-tree-(for-…	STP	60	RST. Root = 32768/0/c4:ad:34:1c:9f:5b Cost = 0 Port = 0x0001
	12 4.824035211	172.16.30.1	172.16.1.10	ICMP	98	Echo (ping) request id=0x0d9c, seq=8/2048, ttl=64 (reply in 13)
	13 4.824520065	172.16.1.10	172.16.30.1	ICMP	98	Echo (ping) reply id=0x0d9c, seq=8/2048, ttl=62 (request in 12)
	14 5.848034732	172.16.30.1	172.16.1.10	ICMP	98	Echo (ping) request id=0x0d9c, seq=9/2304, ttl=64 (reply in 15)
	15 5.848516234	172.16.1.10	172.16.30.1	ICMP	98	Echo (ping) reply id=0x0d9c, seq=9/2304, ttl=62 (request in 14)
	16 6.006329817	Routerboardc_1c:9f:…	Spanning-tree-(for-…	STP	60	RST. Root = 32768/0/c4:ad:34:1c:9f:5b Cost = 0 Port = 0x0001
	17 6.872040903	172.16.30.1	172.16.1.10	ICMP	98	Echo (ping) request id=0x0d9c, seq=10/2560, ttl=64 (reply in 18)
	18 6.872479241	172.16.1.10	172.16.30.1	ICMP	98	Echo (ping) reply id=0x0d9c, seq=10/2560, ttl=62 (request in 17)
	19 7.896029208	172.16.30.1	172.16.1.10	ICMP	98	Echo (ping) request id=0x0d9c, seq=11/2816, ttl=64 (reply in 20)
	20 7.896496881	172.16.1.10	172.16.30.1	ICMP	98	Echo (ping) reply id=0x0d9c, seq=11/2816, ttl=62 (request in 19)
	21 8.008419236	Routerboardc_1c:9f:…	Spanning-tree-(for-…	STP	60	RST. Root = 32768/0/c4:ad:34:1c:9f:5b Cost = 0 Port = 0x0001
	22 8.920030587	172.16.30.1	172.16.1.10	ICMP	98	Echo (ping) request id=0x0d9c, seq=12/3072, ttl=64 (reply in 23)
	23 8.920602746	172.16.1.10	172.16.30.1	ICMP	98	Echo (ping) reply id=0x0d9c, seq=12/3072, ttl=62 (request in 22)
	24 9.948044357	172.16.30.1	172.16.1.10	ICMP	98	Echo (ping) request id=0x0d9c, seq=13/3328, ttl=64 (reply in 25)
	25 9.948467609	172.16.1.10	172.16.30.1	ICMP	98	Echo (ping) reply id=0x0d9c, seq=13/3328, ttl=62 (request in 24)
	26 10.0105414269	Routerboardc_1c:9f:…	Spanning-tree-(for-…	STP	60	RST. Root = 32768/0/c4:ad:34:1c:9f:5b Cost = 0 Port = 0x0001
	27 10.968033249	172.16.30.1	172.16.1.10	ICMP	98	Echo (ping) request id=0x0d9c, seq=14/3584, ttl=64 (reply in 28)

Tux3 ping NetLab

5. DNS

In the fifth experience, we configured DNS on tuxY3, tuxY4, and tuxY2 to utilize the DNS server at services.netlab.fe.up.pt (10.227.20.3). This setup allowed us to resolve hostnames into IP addresses. Connectivity was verified using tools like `ping` and a browser to check if names could be resolved successfully. We also monitored DNS-related packets in Wireshark by executing pings to internet hostnames, observing the DNS query and response packets exchanged.

This experiment highlighted how to configure DNS on a host and provided insights into the types of packets exchanged during DNS queries and the information they carry.

The commands used in this experience were:

cat /etc/resolv.conf
ping github.com

	Time	Source	Destination	Protocol	Length	Info
	2 0.000015924	ASUSTekCOMPU_b3:e9:…	Broadcast	ARP	60	Who has 192.168.109.115? Tell 192.168.109.113
	3 0.000017949	ASUSTekCOMPU_b3:e9:…	Broadcast	ARP	60	Who has 192.168.109.114? Tell 192.168.109.113
	4 0.000019695	ASUSTekCOMPU_b3:e9:…	Broadcast	ARP	60	Who has 192.168.109.112? Tell 192.168.109.113
	5 0.000021442	ASUSTekCOMPU_b3:e9:…	Broadcast	ARP	60	Who has 192.168.109.111? Tell 192.168.109.113
	6 0.140728817	10.227.20.34	10.227.20.3	DNS	69	Standard query 0x04e7 A WORKGROUP
	7 0.140738875	10.227.20.34	10.227.20.3	DNS	69	Standard query 0xa7f0 AAAA WORKGROUP
	8 0.141256475	10.227.20.3	10.227.20.34	DNS	144	Standard query response 0x04e7 No such name A WORKGROUP SOA a.root-servers.net
	9 0.141271002	10.227.20.3	10.227.20.34	DNS	144	Standard query response 0xa7f0 No such name AAAA WORKGROUP SOA a.root-servers.net
	10 0.141425424	10.227.20.34	10.227.20.255	NBNS	92	Name query NB WORKGROUP<20>
	11 0.198501795	10.227.20.34	140.82.121.4	ICMP	98	Echo (ping) request id=0x123e, seq=20/5120, ttl=64 (reply in 12)
	12 0.242410817	140.82.121.4	10.227.20.34	ICMP	98	Echo (ping) reply id=0x123e, seq=20/5120, ttl=46 (request in 11)
	13 0.780083788	ASUSTekCOMPU_2e:20:…	Broadcast	ARP	60	Who has 192.168.109.126? Tell 192.168.109.123
	14 0.780103763	ASUSTekCOMPU_2e:20:…	Broadcast	ARP	60	Who has 192.168.109.125? Tell 192.168.109.123
	15 0.780105579	ASUSTekCOMPU_2e:20:…	Broadcast	ARP	60	Who has 192.168.109.124? Tell 192.168.109.123
	16 0.780107115	ASUSTekCOMPU_2e:20:…	Broadcast	ARP	60	Who has 192.168.109.122? Tell 192.168.109.123
	17 0.780108652	ASUSTekCOMPU_2e:20:…	Broadcast	ARP	60	Who has 192.168.109.121? Tell 192.168.109.123
	18 1.024529038	ASUSTekCOMPU_b3:e9:…	Broadcast	ARP	60	Who has 192.168.109.111? Tell 192.168.109.113
	19 1.024543076	ASUSTekCOMPU_b3:e9:…	Broadcast	ARP	60	Who has 192.168.109.112? Tell 192.168.109.113
	20 1.024544962	ASUSTekCOMPU_b3:e9:…	Broadcast	ARP	60	Who has 192.168.109.114? Tell 192.168.109.113
	21 1.024546708	ASUSTekCOMPU_b3:e9:…	Broadcast	ARP	60	Who has 192.168.109.115? Tell 192.168.109.113
	22 1.024548384	ASUSTekCOMPU_b3:e9:…	Broadcast	ARP	60	Who has 192.168.109.116? Tell 192.168.109.113
	23 1.144660762	10.227.20.34	10.227.20.255	NBNS	92	Name query NB WORKGROUP<1d>
	24 1.200464399	10.227.20.34	140.82.121.4	ICMP	98	Echo (ping) request id=0x123e, seq=21/5376, ttl=64 (reply in 25)
	25 1.245429015	140.82.121.4	10.227.20.34	ICMP	98	Echo (ping) reply id=0x123e, seq=21/5376, ttl=46 (request in 24)
	26 1.292207574	fe80::222:64ff:fea6…	ff02::fb	MDNS	180	Standard query 0x0000 PTR _ftp._tcp.local, "QM" question PTR _nfs._tcp.local, "QM" question PTR _afpovertcp._tcp.local…
	27 1.292299975	10.227.20.34	224.0.0.251	MDNS	160	Standard query 0x0000 PTR _ftp._tcp.local, "QM" question PTR _nfs._tcp.local, "QM" question PTR _afpovertcp._tcp.local…
	28 1.350667519	Cisco_37:79:0a	Spanning-tree-(for-…	STP	60	RST. Root = 32768/0/4c:00:82:2e:9a:00 Cost = 8 Port = 0x800a
	29 1.380571599	ASUSTekCOMPU_b3:e9:…	Broadcast	ARP	60	Who has 192.168.109.13? Tell 192.168.109.113

DNS Github

6. TCP connections

In the sixth experience, we focused on analyzing TCP connections. A custom download application was compiled and executed on tuxY3 while capturing traffic using Wireshark. The goal was to verify that the downloaded file arrived correctly by examining the captured packets. This exercise provided practical experience with TCP connections and packet-level analysis using Wireshark.

Conclusion

This project provided a comprehensive exploration of FTP client development and network configuration, reinforcing key concepts in networking and programming. In Part 1, the development of the FTP client allowed for hands-on experience with the FTP protocol, socket programming, and the intricacies of client-server communication. By implementing functionalities like URL parsing, passive mode, and file transfer, we gained valuable insights into protocol implementation and the challenges of TCP/IP programming.

In Part 2, the focus on network configuration and analysis enriched our understanding of networking principles and protocols. Through experiments made in FEUP'S laboratory involving IP configuration, bridging, routing, NAT, DNS, and TCP connections, we explored various aspects of network design and troubleshooting.

Overall, this project demonstrated the interplay between application development and network configuration in achieving functional and efficient communication systems. The hands-on experiences solidified our knowledge in the subject of computer networks.